

METHODS — כל כלי סטטיסטי בפרויקט, בעברית פשוטה

עודכן: 13.9.2026 · הועבר לספק market

למה זה קיים: יש בפרויקט הרבה מבחנים, וקל לאבד את החוט. המסמך הזה עונה על שלוש שאלות לכל כלי — מה הוא שואל · למה הוא ולא אחר · איפה הוא בקוד.
לא תיאוריה. רק מה שאנחנו משתמשים בו בפועל.

מה השתנה מאז גרסת יום 7. `COST_SPEC=market` הוא הייצור (ADR 0001), ולכן מודל העלות כאן הוא `cost_market.py` ולא `cost_model.py`, והמספרים שלו הוחלפו. נוספו שלושה כלים שלא היו: אילוץ זהות הכדור, אילוץ צורת הדקות (ADR 0005), ו-קונבנציית הניקוד (ADR 0005/0006) — שהיא הכללה של השיעור המרכזי של יום 7. `METHODS.pdf` בשורש נוצר מהגרסה הקודמת ולא מזו.

0. מפה מהירה

כלי	מה הוא עונה	איפה
OLS / רגרסיה	כמה X מסביר את Y	<code>cost_market.py</code>
אפקטים קבועים	להוציא הבדלי רמה בין מועדונים ועונות	<code>cost_market.py</code>
טרנספורמציות log	להפוך יחסים כפליים לחיבוריים	<code>cost_market.py</code>
WLS (משוקלל)	רגרסיה כשלא כל תצפית שווה	<code>production_model.py</code>
GLM בינומי	רגרסיה כש-Y הוא שיעור בין 0 ל-1	<code>avaliability_model.py</code>
התכווצות	לרסן אומדנים ממדגם קטן	<code>roster_optimizer.py</code>
כיול	האם התחזיות ברמה הנכונה	הכל
OOS	האם זה עובד על מה שלא ראינו	<code>Oos_hapoe1.py</code>
מונטה קרלו	להריץ עונה 400 פעם	<code>roster_optimizer.simulate</code>
פיזור-יתר / בטא-בינומי	האם השיעור עצמו לא ידוע	<code>avail_uncertainty.py</code>
תכנון לינארי בשלמים (PuLP)	לבחור סגל תחת אילוצים	<code>optimise_consistent.py</code>
סווייפ רגישות	כמה התשובה תלויה בהנחה	<code>depth_sweep.py</code>
אילוץ זהות	לאכוף זהות שחייבת להתקיים	<code>usage_constrained.py</code>
אילוץ על צורה	לחסום צירוף קצוות, לא כל קצה לחוד	<code>scoring.py</code>
קונבנציית ניקוד	על איזה מידע כל צד עומד	<code>scoring.py</code>
תחזית נעולה מראש	למנוע התאמה בדיעבד	<code>Oos_hapoe1.py</code> ואילך

1. מה שכבר הכרת

רגרסיה (OLS)

$$\log(\text{salary}_i) = a_0 + g_{\text{עונה}} + b1 \cdot \text{pir_lag_shrunk} + b2 \cdot \text{e1_seasons}$$

מה שואלים: בהינתן PIR של אשתקד, מה השכר הצפוי.

מה שקוראים: β_1 הוא הכיוון והעוצמה. p הוא הסיכוי לקבל תוצאה כזו במקרה — קטן מ-0.05 נחשב "לא מקרי". R^2 הוא כמה מהשונות הוסברה.

**אצלנו, תחת market **: $\beta_1 = 0.1727$, $R^2 = 0.953$ מ- refit_acceptance.csv.

ולמה זה הקואפיצינט היחיד שחשוב. כל השאר במודל העלות — החותך, ה- smear, אפקטי המועדון, ונרמול ממוצע המאגר — מכפילים את המחיר של כל שחקן באותו גורם. הכפלת כל העלויות והתקציב באותו גורם משאירה את הקבוצה הפיזיבילית של ה-LP כמו שהיא, ולכן אינה יכולה לשנות מי נבחר. β_1 קובע כמה תלול המחיר עולה עם תפוקה, והוא נושא כמעט את כל פיזור המחירים.

— המספרים שהיו כאן — $R^2 = 0.953$, $\beta_1 = 0.252$, $p = 0.016$ על מכבי ו-0.709 על הפועל — הם כיוול מועדון בודד מלפני ADR 0001. הם אינם מתארים את הייצור.

טרנספורמציות log

למה: שכר גדל כפולית — מ-300K ל-600K זה "פי 2", כמו מ-3M ל-6M. רגרסיה רגילה מניחה תוספת קבועה. log הופכת כפל לחיבור, ואז הרגרסיה מתאימה. מחיר: כשחוזרים ל-exp מקבלים חציון, לא ממוצע. לכן יש smear בקוד — תיקון שמחזיר אותנו לממוצע.

אפקטים קבועים

הבעיה: מכבי משלמת אחרת מ-ASVEL על אותו שחקן, ו-2019 אחרת מ-2025. בלי להוציא את הבדלי הרמה האלה, β_1 סופג אותם ומודד "אילו מועדונים עשירים" במקום "כמה תפוקה מייקרת שחקן".

הפתרון: מקדם נפרד לכל מועדון ולכל עונה. ההשוואה נעשית בתוך מועדון-עונה.

מה זה קונה: אפשר לתמחר שחקן ב"מועדון ממוצע בליגה" במקום במועדון הכיול. וזו הסיבה ש-cost חסר יחידות — המחיר מחולק בממוצע המאגר של העונה, כך שיחידה אחת היא "שחקן מאגר ממוצע אחד". **אין לקרוא cost כסכום כספי. **ציר היורו נבדק ונדחה, ראו docs/refit-day15-diff.md.

מחיר שנרשם: הפיצול לכיול ולבדיקה הוא לפי מועדון ולא לפי שורה. מועדון שלם הולך לצד אחד, כי אפקט קבוע של מועדון שנאמד על אותן שורות שבדקות אותו אינו מבחן חוץ-מדגמי.

משתנה קטגוריאל

e1_seasons — מספר עונות ביורוליג. אפשר להתייחס אליו כמספר רציף (כל עונה מוסיפה אותו דבר) או כקטגוריות (עונה 1, 2, 3, 4 נפרדות).

איפה זה עומד: ספק ה- `market` רץ רציף, עם קואפיצינט אחד β_2 .
`ro.COST_FEATURES` הוא בדיוק שתי תכונות — `pir_lag_shrunk` ו- `el_seasons` — ולא
שבע. המחלוקת אינה מוכרעת ב-ADR; היא פשוט אינה חיה, כי הנוסחה שרצה בייצור היא זו
שלמעלה. $\ominus$ הנוסחה הארוכה שהופיעה כאן וב- `project_state.md`, `age_c`, `age_c^2`,
`min_lag` ודאמיז ל- `el`, אינה מה שרץ.

סטיית תקן

הפיזור סביב הממוצע. החשוב: אנחנו מבחינים בין שני סוגים —

	מה זה	דוגמה
שארית	הפיזור האמיתי בין שחקנים	שחקנים באמת שונים זה מזה
שגיאת אמידה	כמה אנחנו לא בטוחים בתחזית	מדגם קטן → לא בטוחים

זה היה באג ביום 6: ענשנו שחקנים לפי השארית — כלומר הענשנו שחקן שבאמת טוב יותר מהתחזית. אבל הוא
מה שמחפשים. קללת המנצח נובעת משגיאת האמידה. התיקון: מעבר ל- `se_mean`.

2. מה שהוספנו

WLS — רגרסיה משוקללת

הבעיה: שחקן ששיחק 34 משחקים ושחקן ששיחק 3 — ה- `ppm` של הראשון אמין הרבה יותר.
הפתרון: לתת לכל תצפית משקל. אצלנו המשקל הוא סך הדקות.
איפה: `sm.WLS(..., weights=minutes_tot)`, `production_model.py`

GLM בינומי — כש-Y הוא שיעור

הבעיה: זמינות היא משחקים / משחקים אפשריים — תמיד בין 0 ל-1. רגרסיה רגילה יכולה לנבא 1.3 או -0.2, וזה
חסר משמעות.
הפתרון: מודל שמובנה כך שהתוצאה לא יכולה לצאת מהתחום. הקשר הוא `logit`:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

הצד השמאלי חופשי לנוע מ- $-\infty$ ל- $+\infty$, והצד הימני תמיד בין 0 ל-1.

אצלנו: `logit(frac) = -0.962 + 0.056 \cdot \text{גיל} - 0.008 \cdot \text{זמינות\_קודם} + 1.456 \cdot \text{דקות\_קודם}`
איפה: `avaliability_model.py`

התכווצות (Shrinkage)

הבעיה: שחקן ששיחק 3 משחקים עם PIR 20 — האם הוא באמת כזה, או שהיה לו מזל?
הפתרון: למשוך את האומדן שלו לכיוון ממוצע הליגה, ככל שיש עליו פחות דאטה.

$$w = \frac{k}{k + \text{משחקים}} \quad k = 40$$

$$\text{ממוצע_הליגה} = w \cdot \text{שלו} + (1-w) \cdot \text{אומדן}$$

3 משחקים $\rightarrow w = 0.07$, כלומר 93% ממוצע ליגה. 34 משחקים $\rightarrow w = 0.46$.
זה פרמטר מוצהר, ו- k נסרק ב-20 ו-60 כבדיקת רגישות.

כיוול — ולמה הוא חשוב יותר מדירוג

דירוג: האם סידרנו את השחקנים נכון.
כיוול: האם הרמה נכונה.

מודל שאומר "כולם יהיו זמינים 90%" כשבפועל זה 75% — מדרג מושלם ומשקר על הרמה. וכשמכניסים אותו לאופטימיזר שמחלק תקציב, כל ההקצאה שגויה.

איך בודקים: מחלקים את השחקנים לעשירונים לפי התחזית, ובכל עשירון משווים תחזית ממוצעת מול מה שקרה. הפער המרבי הוא הציון.

אצלנו: זמינות — פער 0.054 מול סף 0.070 שנרשם מראש ☒ · תפוקה — 0.096 מול 0.05 ☒ נכשל

OOS — מחוץ למדגם

הרעיון: לאמן על 2023 ומטה, לנבא את 2024, ולהשוות למה שקרה. מודל שנראה מצוין על הדאטה שלו ונופל מחוץ לו — התאים את עצמו לרעש.

אצלנו: זמינות — 0.753 חזוי מול 0.740 בפועל. פער $+0.013$ ☒

מונטה קרלו

הרעיון: במקום לחשב "מה יקרה בממוצע", להריץ את העונה 400 פעם עם הגרלות ולראות את ההתפלגות. מה מוגרל אצלנו, ובאיזה סדר:

השחקן הוא מה שהוא — (תחזית, סטיית_תקן) $\sim N(\text{ppm})$: פעם אחת לעונה
בכל משחק : נוכח / לא נוכח

הסדר הזה הוא כל העניין. אם הכוכב מתגלה חלש מהצפוי, זה נכון לכל העונה, וצריך למי להעביר את הדקות. זה מה שנותן לעומק ערך, ומה שהחישוב הדטרמיניסטי לא רואה.

פיזור-יתר ובטא-בינומי

השאלה: ה-GLM נותן "לשחקן הזה 78% זמינות". אבל כמה אנחנו בטוחים במספר הזה עצמו?

המבחן: אם המודל מושלם, הסטייה של כל תצפית מהתחזית צריכה להיות בגודל שהמודל הבינומי צופה. מחשבים

ממוצע $\phi = [(\text{בפועל} - \text{חזוי})^2 / \text{השונות_הצפויה}]$

$\phi = 1 \rightarrow$ אין אי-ודאות נוספת. $\phi = 10.38$ (מה שקיבלנו) $\rightarrow$ השונות גדולה פי עשרה ממה שהמודל מניח.

התיקון — בטא-בינומי: קודם מגרילים את השיעור האמיתי של השחקן מהתפלגות בטא סביב התחזית, ורק אז מגרילים משחק-משחק.

אצלנו: ס"ת של השיעור האמיתי סביב 0.78 הוא 0.228 — שחקן שנחזה 78% הוא בפועל בטווח 55%-100%.

איפה: `avail_uncertainty.py`

תכנון לינארי בשלמים (PuLP)

מה זה: בחירה תחת אילוצים. משתני החלטה בינאריים (x_i = בסגל או לא) ורציפים (e_i = דקות), ומקסמים ביטוי לינארי.

```
max  $\sum \text{ppm}(i) \cdot e(i)$ 
 $\sum e(i) \leq 200$  •  $e(i) \leq 32 \cdot x(i)$ 
 $\sum \text{עלות}(i) \cdot x(i) \leq \text{תקציב}$ 
 $12 \leq \sum x(i) \leq 16$  • רצפות ותקרות עמדה
 $\sum e(i) \cdot (\text{usage}(i) - 20) \leq 0$  • זהות הכדור, מסלול הכותרת
 $\sum e(i) \leq C_k$  • עד 8 צורת הסבב, לכל k (ADR 0005)
```

הכלל הקריטי שנלמד ביום 7: הפונקציה שממקסמים חייבת להיות בדיוק הפונקציה שמנקדת. אם לא — הידוק אילוח יכול להעלות את הערך, וזה בלתי אפשרי מתמטית. זו הייתה ההוכחה לבאג.

אילוח זהות — כשמספר חייב להתקיים

הרעיון: לא כל אילוח הוא הנחה. `usage` הוא נתח ההחזקות מתוך אלה של הקבוצה, והממוצע המשוקלל-דקות שלו חייב להיות 20.0% — חמישה על המגרש, כדור אחד. זו זהות, לא אמידה, ולכן אין לה סף להצדיק.

$$\sum e(i) \cdot \text{usage}(i) \leq 20 \cdot \sum e(i) \quad \Leftrightarrow \quad \sum e(i) \cdot (\text{usage}(i) - 20) \leq 0$$

ליניארי לגמרי, בלי משתנים חדשים. הסתייגות שנרשמת: `usage` נלקח מהעונה הקודמת, כי זה מה שידוע בזמן הבחירה. כלומר האילוח אוכף שהסגל היה עומד בזהות אילו כל שחקן שמר על נתחו — נכון כאילוח תכנוני, לא כניבוי.

אילוח על צורה — לא כל קצה לחוד

הבעיה, והיא חזרה שלוש פעמים. תקרת 32 הדקות חלה על כל שחקן בנפרד, ולכן האופטימיזר נותן אותה לכולם בו-זמנית: 192 דקות לשישייה, מול 145.3 של הקבוצה הכי מרוכזת ב-38 עונות-מועדון. אותו דבר קרה עם רצפות העמדה. בשני המקרים לקחנו קצוות שוליים, כל אחד מקבוצה אחרת, והרכבנו מהם קבוצה שאיש לא שיחק בה. אופטימיזר תמיד יילך לפינה.

הפתרון: לחסום את סכום k הגדולים לכל k, ולא כל שחקן לחוד. זה ניתן לביטוי ליניארי בזהות תקנית (Nesterov), ולכן נכנס ל-LP בלי לשבור אותו:

$$e \leq C_k \text{ הגדולים של } k \text{ סכום}$$
$$\Leftrightarrow k \cdot q + \sum s(i) \leq C_k, \quad s(i) \geq e(i) - q, \quad q \geq 0$$

מה נמדד: האילוח כובל, ומוריד את הכותרת ב-14.8%. ה- k שכובלים הם 4 עד 8 ($k=8$ ב-97% מהפתרונות), ו- $k=1, 2, 3$ לא כובלים אף פעם — ההנחה הקודמת ש"א סמוכים כמעט מיותרים" הייתה שגויה בשני הכיוונים.

איפה: `scoring.add_shape_constraint` — מימוש אחד שאותו קוראים שני האופטימיזרים. שני מימושים לאילוץ אחד הוא איך שנוצרו שתי פונקציות ניקוד.

קונבנציית ניקוד — על איזה מידע כל צד עומד

ההכללה של השיעור מיום 7. שם למדנו שהמטרה חייבת להיות פונקציית הניקוד. הכלל הרחב יותר הוא ששני צדי ההשוואה חייבים לעמוד על אותו מידע.

הסקורר המקורי מחלק דקות בסדר יורד של `ppm_true` — כלומר בדיעבד, בידיעה מי יצא טוב. כל זמן ששני הצדדים קיבלו את זה, זו הייתה סימטריה. אבל סימטריה שמייפה את המועדון: היא מעניקה לשישייה שלו 177.0 דקות במקום 125.7 ששיחקה, מכנה מנופח ב-40.8%.

א. שני הצדדים מחולקים מחדש בדיעבד סימטרי • מייפה את המועדון • 0.1414
(ADR 0005) ב. המועדון על מה ששיחק, המנוע בדיעבד א-סימטרי • 98.4%+ • נעצר
התוצאה טרם נמדדה • ADR 0006 ג. שני הצדדים על מה שהתחייבו אליו

(ב) נעצר בכלל שהוצהר מראש. הוא מערבב את הטענה של הפרויקט — בחירת סגל טובה יותר באותו תקציב — עם טענה שהוא אינו טוען: ניהול דקות טוב יותר ממאמנים, עם ידיעת התוצאה.

****ג) מנקד את המנוע על $\min(e_{LP}, 32 \cdot \text{זמיןות בפועל})$ — התוכנית שהתחייב אליה, חתוכה לפי הזמינות שהתממשה — ואת המועדון על הדקות ששיחק. הדקות שהתפנו אינן מחולקות מחדש; הן נופלות לרמת המחליף, וזה מחיר הסיכון של לתכנן סביב שחקן שלא היה שם. נמדד: חציון 27.3** דקות מ-200 נקצצות כך.**

סווייפ רגישות

מתי: כשמספר בקוד הוא הנחה ולא מדידה.
מה עושים: מריצים על פני טווח שלם ומדווחים את הטווח, לא ערך אחד.
דוגמה חיה: רמת המחליף. מדדנו 0.127, אבל היא נעה בין 0.030 ל-0.179 לפי הסף שבוחרים. לכן היא נסרקת ולא נבחרת.

תחזית נעולה מראש

לא כלי סטטיסטי — כלל עבודה, והוא החשוב ביותר בפרויקט.
לפני שמריצים, כותבים בראש הקובץ מה אנחנו צופים ולמה. אחר כך אי אפשר להתאים את הפרשנות לתוצאה.
זה עבד: ביום 6 חזיתי שאצל הפועל התקציב לא מגביל. ניצול היה 99.7%. התחזית נפלה, וזה היה הממצא.
זה נשמט: בבוקר יום 7 הרצתי שלושה קבצים בלי אף תחזית. הבוקר הזה הוא היחיד שכל מסקנותיו נפלו.

3. הכשלים החוזרים — מה לבדוק לפני שמדווחים מספר

הכשל	דוגמה בפרויקט
שני צדי ההשוואה אינם אותם אנשים	בנצ'מרק מכבי — 9 שחקנים במקום 12, בלי שני הטובים
סקלה מעונה אחרת	תמחור מועמדי 2024 לפי שכר ממוצע של 2023
מכנה לא נבדק	לויז — 22.8 דק' למשחק אחרי משחק אחד
מזהה נחשב למספר	Player_ID , player_code , gamecode — שלוש פעמים
מבחן שלא יכול להראות אחרת	בחירת גודל סגל לפי ניקוד שמונוטוני יורד
כישלון שקט	Skip and continue · .gitignore שחוסם דאטה
סינון חסר	בוקסקור כולל פלייאוף, player_season לא
** inner join על עמודה ריקה**	בקרת התקציב רצה על gross_eur , ריק ב-20/20 מועדוני 2025 — כל העונה נשרה בשקט
סף שנוסח לציר אחד ונבדק על אחר	תנאי 1 ב- refit_acceptance בחן a=1.443 מהציר המנורמל, מול סף שנוסח ל- a=1.087 ברמת היורו
עמודה שמבטיחה דאטה שאינו שם	minutes_alloc ריק ב-456/456, כי ההיוריסטיקה חיפשה מערך בערך ההחזרה של score_rows — שמחזירה שלושה סקלרים
התניה חסרה	מיצוע דקות כולל משחקים שבהם השחקן נעדר

ארבעה מתוך שמונה הם אותה משפחה: דיווחתי מספר לפני שווידאתי ששני הצדדים עומדים על אותו בסיס.

4. איפה כל מבחן יושב

קובץ	מה הוא בודק
cost_market.py	● מודל העלות של הייצור — OLS עם אפקטי מועדון ועונה
cost_model.py	הספק הקודם, club_relative. קיים רק לשחזור מספרי יום 14
production_model.py	WLS — ppm צפוי
avaliability_model.py	GLM בינומי — שיעור משחקים
audits/survival_model.py	שרידות (הוחלף, לא נזרק)
Oos_hapoel.py	מבחן חוץ-מדגמי עם תחזיות נעולות
league_jump_test.py	הליגה הישראלית — הטיית בחירה
roster_optimizer.py	PuLP + מונטה קרלו
optimise_consistent.py	● יישור המטרה לניקוד
optimizer_backtest.py	הבקטסט של המנוע
backtest_diagnostics.py	שלוש בדיקות על המבחן
roster_membership_audit.py	מי נחשב לסגל
benchmark_matched.py	ההשוואה מול המועדון
curse_decomp.py	פירוק קללת המנצח
replacement_level.py	מידת רמת המחליף
depth_sweep.py	סווייפ רגישות
split_multiclub.py	פיצול לפי מועדון מבוקסקור
avail_uncertainty.py	פיזור-יתר בזמינות
curse_redistribution.py	קללה — ממצא או ארטיפקט
absence_position_test.py	לאן דקות הנעדר הולכות
usage_constrained.py	● מסלול הכותרת — אילוץ זהות הכדור
scoring.py	אילוץ הצורה וקונבנציות הניקוד (ADR 0005/0006)
shape_constraint_test.py	הטסטים של אילוץ הצורה
league_backtest.py	הבקטסט על 38 עונות-מועדון
fatigue_delta.py	אמידת δ העייפות — הופרך על ראיות (ADR 0004)
null_to_wins.py	מודל האפס
wins_conversion.py	המרת ניקוד לניצחונות

קובץ	מה הוא בודק
refit_acceptance.py	שערי הקבלה של הריפיט
refit_diff.py	טבלת לפני/אחרי, מקבצי תוצאה מקומטים
final_day7.py	ההרצה המסכמת